

مساعد يرد على أسئلة المواصفات: الدراسة كاملة.

١٢ مرحلة. الخطوات، المعادلات، ومعنى كل مصطلح.



إزاي تتأكد إن فيه مشكلة أصلًا

- 1 اقعد جنب المكتب الفني أسبوعين وسجّل كل سؤال بيتسأل عن المواصفة.
- 2 لكل سؤال اكتب: مين سأل، وإمتى، وقد إيه استنى لحد ما جاله رد.
- 3 علّم الأسئلة اللي اتكررت. التكرار هو الدليل، مش شكوى الناس.
- 4 لو العدد طلع صغير، اقفل الملف هنا واكتب السبب في ورقة.



حساب حجم المشكلة بالفلوس

$$pain = f \times t \times n \times loaded\ rate$$

- 1 f: عدد المرات في السنة، من العدّ اللي عملته في المرحلة اللي فاتت.
- 2 t: متوسط الوقت الضايع في المرة الواحدة، بالساعة.
- 3 n: عدد الناس اللي شغلهم بيقف لحد ما السؤال يتحل.
- 4 زوّد تكلفة الإجابة الغلط: عدد مرات الخطأ × تكلفة تصحيحه.

loaded rate — تكلفة ساعة الموظف على الشركة: المرتب زائد التأمينات والأوفرهيد، مش المرتب لوحده.

قرار: نبني، نشترى، ولا نوقف

- 1 اسأل: الشغلانة دي بتميز الشركة قدام العميل؟ لو لأ، متبنيش.
- 2 دوّر على منتج جاهز بيعملها. لو موجود ومقبول، اشترى.
- 3 لو الرقم من المرحلة ٢ صغير، وقّف. ده قرار سليم مش فشل.
- 4 اكتب القرار وسببه في ورقة، عشان يتراجع بعد ٦ شهور.



تحديد الخطوة اللي هتتغير

- 1 ارسم الـ as-is: الخطوات الحالية من لحظة السؤال لحد الرد.
- 2 ارسم الـ to-be: نفس الرسمة بعد ما الأداة تشتغل.
- 3 علّم الخطوة الوحيدة اللي اتغيرت. لو أكثر من واحدة، النطاق واسع.
- 4 سقي مين هيبقى مسؤول عن الناتج بعد التغيير.

as-is / to-be — الوضع الحالي والوضع المستهدف، مرسومين جنب بعض على نفس الورقة.



حساب العائد على الاستثمار

$$ROI = (value - cost) / cost$$

- 1 value: الساعات الموفرة $\times$ loaded rate, زائد الأخطاء التي اتمنعت $\times$ تكلفتها.
- 2 cost: البناء + الـ tokens + وقت الـ reviewer + وقت تدريب الناس.
- 3 الأداة تنتقل الشغل من المنقذ للمراجع, فوقت الـ reviewer تكلفة حقيقية.
- 4 لو نسيته من المعادلة, أي فكرة هتطلع رابحة. ده تبرير مش حساب.

token — الوحدة التي شركات الـ AI بتحسب بيها الاستهلاك والفاتورة. تقريبا كلمة أو جزء منها.
reviewer — الشخص الذي يراجع مخرجات الأداة قبل ما حد يستخدمها.



الطبقات الأربعة وترتيبها

- 1 Ingestion: تجهيز المستندات وتقطيعها بشكل يحافظ على البنود.
- 2 Retrieval: البحث الذي يرجع القطع المتعلقة بالسؤال.
- 3 Generation: الموديل، ويشتغل على الذي رجع من Retrieval بس.
- 4 Review queue: طابور بشري لأي إجابة رجعت بثقة ضعيفة.

Ingestion — إدخال المستندات ومعالجتها وتخزينها قبل أي بحث.

Retrieval — استرجاع الأجزاء المتعلقة بالسؤال من المستندات المخزنة.



تقطيع المستند وتخزينه

`chunk = clause, not fixed characters`

- 1 قَطِّع المستند عند حدود البنود، مش كل ٨٠٠ حرف.
- 2 خَزِّن مع كل chunk: اسم المستند، الباب، رقم البند، ورقم المراجعة.
- 3 البيانات دي هي اللي بتخلي الإجابة ليها سند تقدر تراجع.
- 4 لو قَطَّعت بعدد الحروف، البند بينقسم والجملة الحاكمة بتضيع.

chunk — القطعة الواحدة من المستند اللي بتتخزن ويتم البحث جواها.

clause — البند المرقم في المواصفة، زي 08 41 13 - 2.3.



القيود المفروضة بالكود

no clause reference -> reject

- 1 الموديل ممنوع يستخدم أي معلومة برّه اللي رجع من Retrieval.
- 2 كل إجابة لازم ترجع ومعها رقم البند اللي جت منه.
- 3 الكود بيتأكد إن الرقم ده موجود فعلاً جوه النص اللي رجع.
- 4 لو مش موجود، الرد بيترفض ومبيوصلش للمستخدم.
- 5 ده بيتعمل في الكود، مش بتعليمه في ال-prompt.

prompt — التعليمات النصية اللي بتتبع للموديل مع كل سؤال، بتوجه، ومبتفرضش.



الأسئلة الممنوعة من الأصل

- 1 اكتب قائمة بالمواضيع التي الأداة مش هترد عليها خالص.
- 2 الحد الأدنى: التفسير التعاقدي، الأوامر التغييرية، وأي حاجة سلامة.
- 3 التحويل لبني آدم بيتم بقاعدة على الموضوع، مش بدرجة ثقة الموديل.
- 4 اعرض القائمة على القانوني واعتمدها قبل التشغيل.

Guardrails — قيود ثابتة بتمنع النظام من التصرف في مواقف محددة مسبقًا.



المخاطر ومين مسؤول عن كل واحدة

- 1 المواصفة بتتراجع: إجابة صح على Rev C بتبقى غلط على Rev D.
- 2 التخفيف: اطبع رقم المراجعة على كل إجابة، ونبّه لما البند يتغير.
- 3 اللي مش هيتحل: حد فاتح إجابة قديمة محفوظة عنده. اكتبه كخطر مفتوح.
- 4 كل خطر يتكتب جنبه اسم شخص، مش اسم قسم.



اللي لازم يكون جاهز قبل أول يوم

- 1 **صلاحيات:** مين يشوف مستندات أنهي مشروع.
- 2 **Logging:** كل سؤال وكل إجابة متخزين وتقدر ترجعلهم.
- 3 **سقف إنفاق شهري،** لأن التسعير بالـ token بيتحرك مع الاستخدام.
- 4 **بيئة اختبار منفصلة عن اللي الناس بتستخدمه فعليًا.**

Logging — تسجيل كل عملية بتحصل في النظام عشان تقدر ترجعلها وتراجعها بعدين.



بناء عينة الاختبار وتحديد العتبات

```
accept if recall >= R and precision >= P
```

- 1 اجمع 50 سؤال حقيقي قديم إجاباتهم معروفة ومتأكد منها.
- 2 حدد R و P كأرقام مكتوبة قبل ما تشغل التجربة.
- 3 لو الأخطر إنه يفوت بند، ارفع R. لو الأخطر إنه يخترع، ارفع P.
- 4 أي عتبة بتتحدد بعد ما تشوف النتيجة دي تبرير مش عتبة.

recall — من كل الإجابات الصح الموجودة في المستند، الأداة لقت كام منها.
precision — من كل اللي الأداة رجّعت، كام واحد كان صح فعلاً.



خطة النزول للتشغيل

- 1 مشروع واحد وفريق واحد. مش الشركة كلها.
- 2 سقي شخص مسؤول بالاسم عن إصلاح الإجابات الغلط.
- 3 أي شكوى تتحول أوتوماتيك لحالة جديدة في عينة الاختبار.
- 4 حدد تاريخ مراجعة: بعد ٦ أسابيع نقرر نكمل ولا نوقف.



قياس الاستخدام الحقيقي

$$\text{adoption} = \text{repeat users} / \text{eligible users}$$

- 1 eligible users: كل اللي شغلهم يستدعي الأداة، مش اللي عندهم حساب.
- 2 repeat users: اللي رجعوا في الأسبوع الرابع من غير ما حد يفكرهم.
- 3 اقيس كمان عمق الاستخدام: أسئلة شغل ولا أسئلة تجربة.
- 4 متقيسش الـ logins ولا عدد المقاعد. دي نشاط مش قيمة.

eligible users — الناس اللي طبيعة شغلهم بتستدعي استخدام الأداة فعلاً.



إثبات إن الرقم اتحقق

$$\text{realized} = (\text{base} - \text{actual}) \times \text{rate} - \text{run cost}$$

- 1 base: متوسط الوقت قبل الأداة، من الـ baseline بتاع المرحلة ١.
- 2 actual: نفس القياس بالظبط بعد ٣ شهور تشغيل.
- 3 run cost: تكلفة التشغيل الفعلية في نفس الفترة.
- 4 من غير baseline، مفيش رقم تقدر تدافع عنه قدام الإدارة.

baseline — القياس اللي بتأخذه قبل أي تغيير، عشان يبقى عندك حاجة تقارن بيها.



الترتيب اللي المفروض تمشي بيه

- 1 ابدأ بالمرحلة ١ و ٢ على فكرتك إنت. عدّ الأول، وبعدين احسب.
- 2 لو الرقم طلع صغير، وقّف. دي أرخص نقطة تقف عندها.
- 3 لو كبير، اكتب الـ Architecture قبل ما تختار موديل.
- 4 الموديل آخر قرار، وأسهل قرار تقدر تغيّره بعدين.

